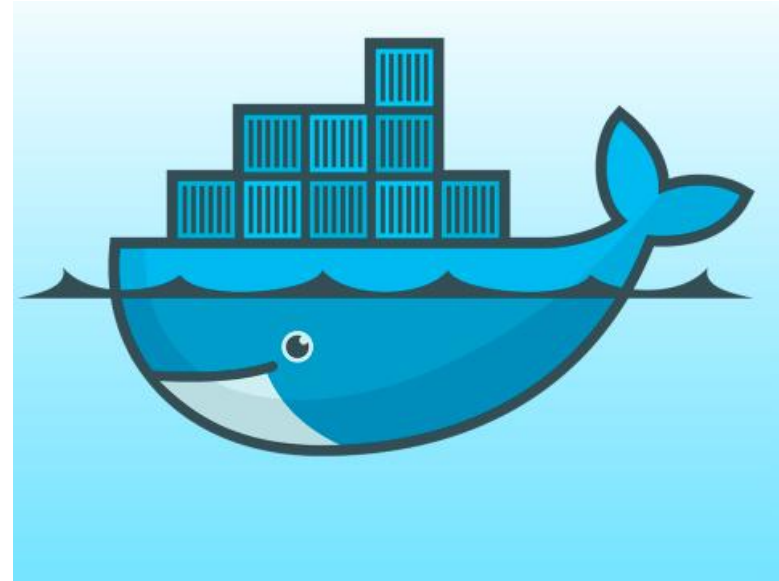
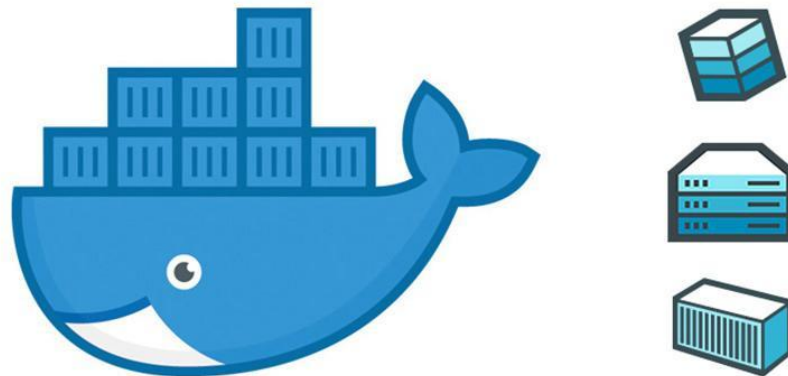


AWS and Docker



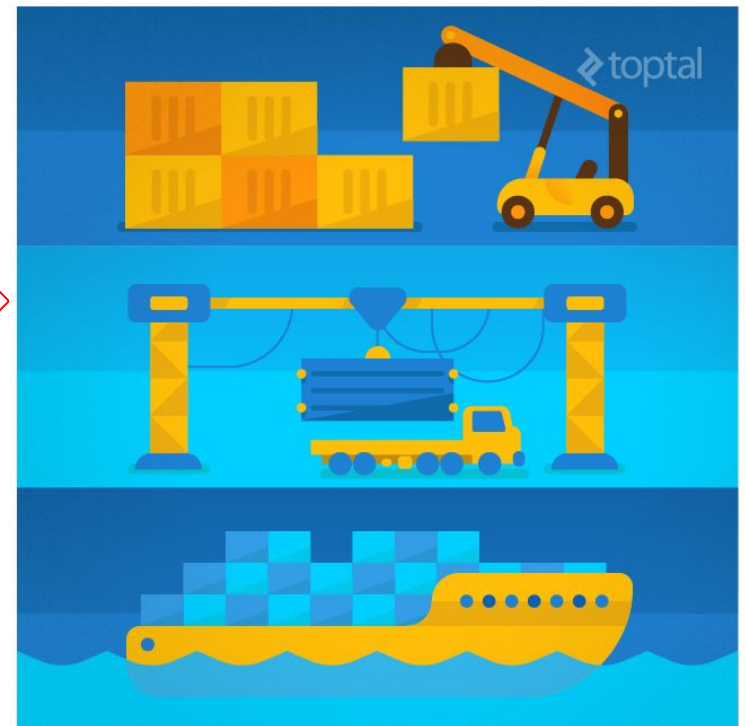
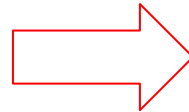
Docker introduction

- Docker is a software container platform.
 - **Developers** use Docker to eliminate “works on my machine” problems when collaborating on code with co-workers.
 - **Operators** use Docker to run and manage apps side-by-side in isolated containers to get better compute density.
 - **Enterprises** use Docker to build agile software delivery pipelines to ship new features faster, more securely and with confidence for both Linux and Windows Server apps



Docker analogy

- Usage of containers in the transportation industry.



Virtual machine

- A computer is an electronic device that creates execute some process.
- Computers are built with hardware components. Hardware components are not possible to use for a regular user. This where to software comes to the picture.
- Here are some examples of software operating systems that can control hardware.



Virtual machine, continue

- A virtual machine manager also called a hypervisor. It's a type of software that allows us to run an operating system within another operating system.
- Here some examples of virtual machines managers.



Virtual machine, example

RUNNING OS X



INSTALL VIRTUALBOX



**WITH IT CREATE TWO
VIRTUAL MACHINES**

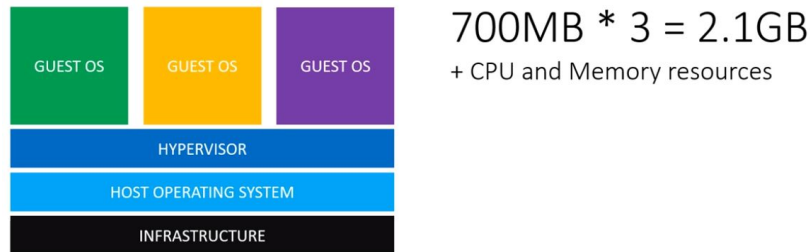


A few reasons for using a virtual machine

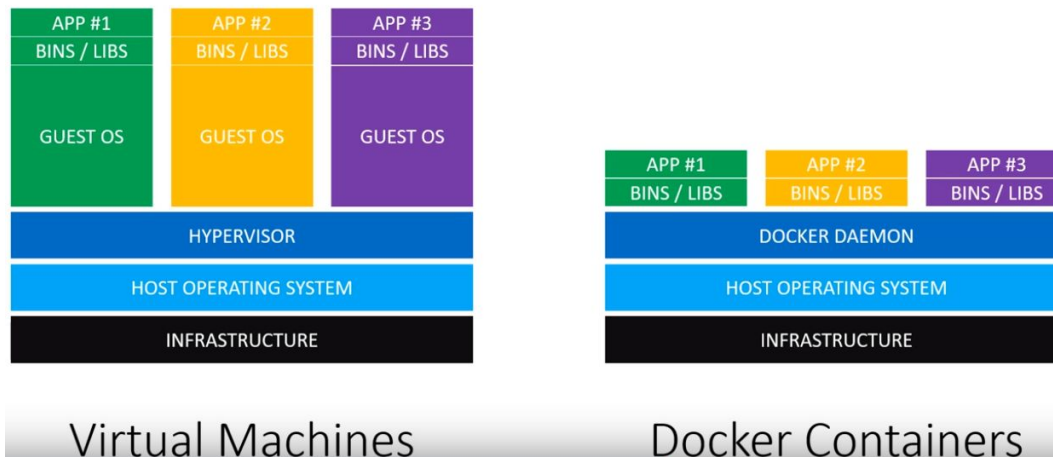
- Trying a new operating system.
- Testing an application.
- Running old applications that are not supported by the current operating system.

Virtual Machines vs Docker Containers

- Docker containers are not virtual machines.
- In Virtual machines, there is a huge waste of memory.



- The difference between the structure of VM and Docker.



Docker Terminology

- **Images**

- A docker image is the "**cooking recipe**" for a container. It contains all the definitions of how to boot up the Linux environment. Typically one Docker image is there to fulfill specifically one task. For instance, one image would define your webserver and another image would define the underlying database.

- **Containers**

- Containers are the **instantiations of images**. They are one form of the image. Compare it to object-oriented programming, then your class would be an image and instance of the class a container. Another comparison would be to real containers on the ship. They have a underbody from the outside but can look very different from the inside. Some interesting facts about containers:
- containers are not persistent. As soon as you don't use them anymore they get stoped (or even deleted)
- a container only lives for the time a process runs inside it which does not get killed (of course you could run processes which never send an exit command)
- containers can include data however it's also not persistent. If you want to make it persistent, you can use volumes.
- to communicate between containers, use the same protocols as you would use to communicate between computers (e.g. tcp/ip)

Docker Terminology continue

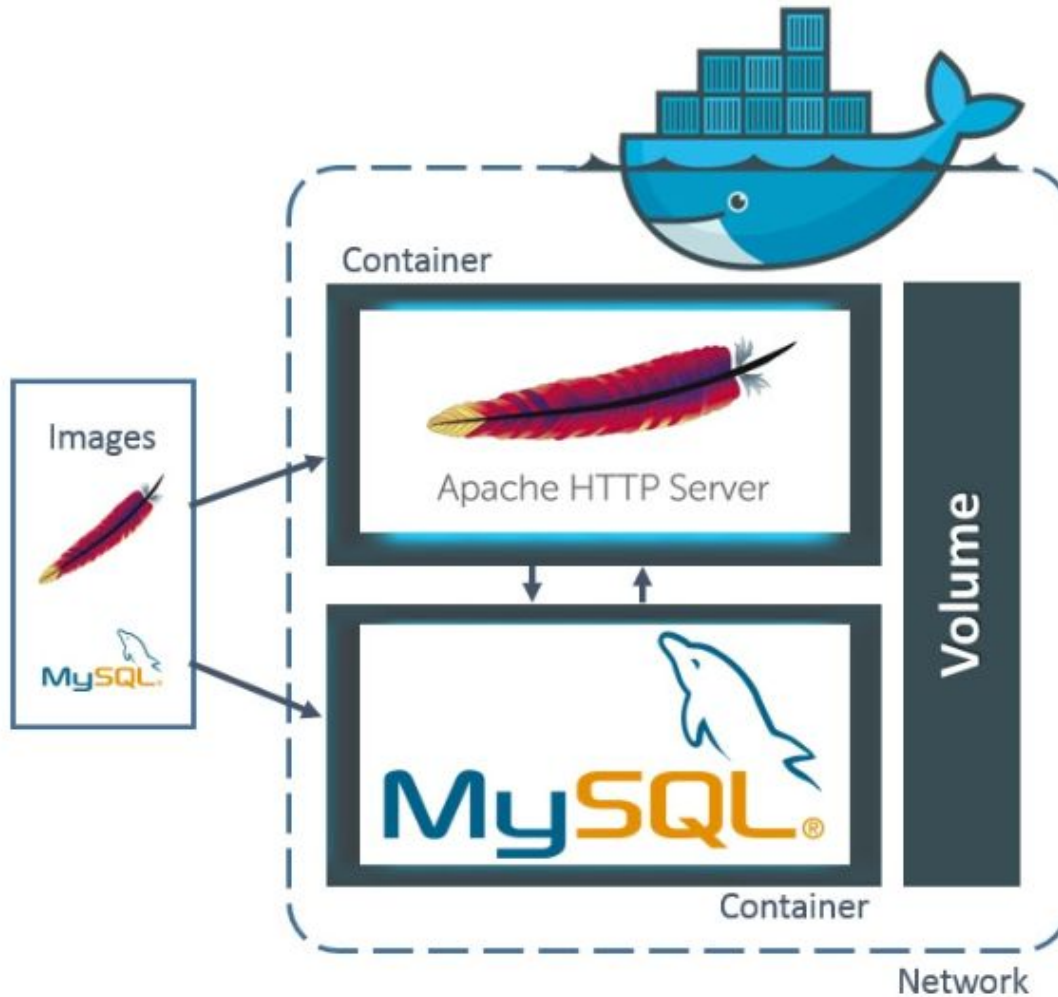
- **Volumes**

- Volumes can be used as the **underlying data layer**. They can be used between as many containers as you want. All data stored in here could be used by all containers connected to it.

- **Networks**

- Docker comes with its networking capabilities. The name of a container is its hostname. The easiest way to try this is to have two containers running and ping one container from inside the other container. Docker Compose will raise its subnet.

A diagram of the Docker



Download Docker

Sign up to Docker site, under the following Url

<https://hub.docker.com/signup?next=%2F%3Fref%3Dlogin:>



Docker Identification

In order to get you started, let us get you a Docker ID.
Already have an account? [Sign In](#)

simonx23

.....



kalex7878@gmail.com



I agree to Docker's [Terms of Service](#).



I agree to Docker's [Privacy Policy](#) and [Data Processing Terms](#).



(Optional) I would like to receive email updates from Docker, including its various services and products.



I'm not a robot

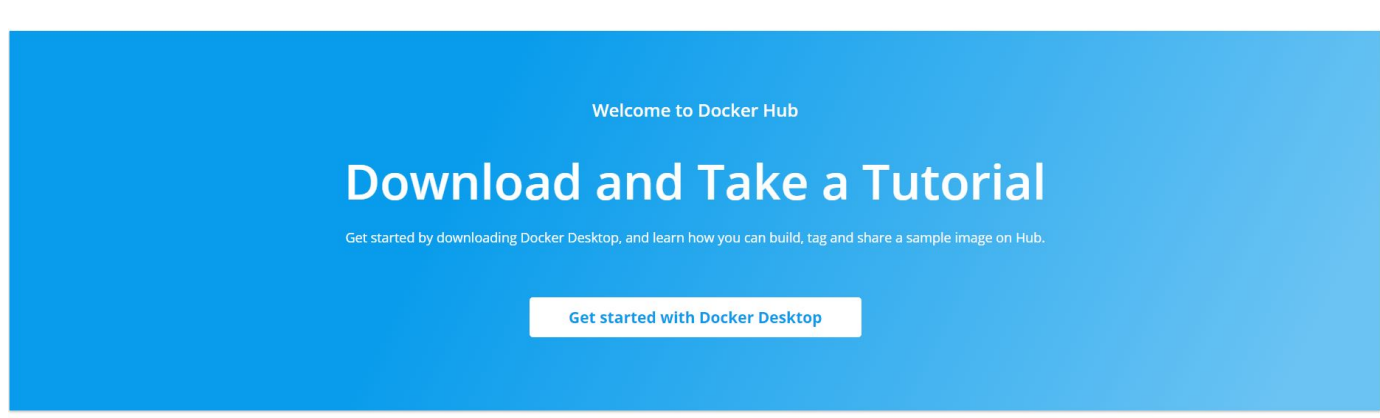


reCAPTCHA
Privacy - Terms

Continue

Download Docker continue

In the home page of the site press on the link



Check Docker installation

Let's check that Docker installed, we will check it in power shell.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\alekx> docker --version
Docker version 19.03.2, build 6a30dfc
PS C:\Users\alekx>
```



Execution of our first Docker command

Let's execute our first Docker command.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\alexx> docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\Users\alexx>
```


Containers information

- You can see the hello world container that we just created in this list.

```
PS C:\Users\alexx> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
1d4f7f4b3b9a	hello-world	"/hello"	About a minute ago	Exited (0) About a minute ago		charming_buck

- The name of the container is just randomly generated. We can refer to the containers by names or ID's.
- Let's start a Docker container with a name that we chose.

```
PS C:\Users\alexx> docker run --name name-example hello-world
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

```
PS C:\Users\alexx> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
f6d2e8031bfc	hello-world	"/hello"	6 seconds ago	Exited (0) 5 seconds ago		name-example
1d4f7f4b3b9a	hello-world	"/hello"	2 minutes ago	Exited (0) 2 minutes ago		charming_buck

Linux bash container

- We can now download a container that allows us to execute Linux from it.

```
PS C:\Users\alexk> docker run -it --name linux-container ubuntu bash
root@372de3e24e55:/#
```

- The # shows us that we are inside the container.
- We can try to execute a single Linux command from the container.

```
root@372de3e24e55:/# echo "Hello Docker"
Hello Docker
root@372de3e24e55:/#
```

- We can see the files inside the container.
- We can exit from the container.

```
root@372de3e24e55:/# exit
exit
```

```
root@372de3e24e55:/# ls -ltr
total 64
drwxr-xr-x  8 root root 4096 May 23  2017 lib
drwxr-xr-x  2 root root 4096 Apr 24  2018 home
drwxr-xr-x  2 root root 4096 Apr 24  2018 root
drwxr-xr-x  1 root root 4096 Aug  7 13:02 usr
drwxr-xr-x  2 root root 4096 Aug  7 13:02 dev
drwxr-xr-x  2 root root 4096 Aug  7 13:02 opt
drwxr-xr-x  2 root root 4096 Aug  7 13:02 net
drwxr-xr-x  2 root root 4096 Aug  7 13:02 media
drwxr-xr-x  2 root root 4096 Aug  7 13:03 lib64
drwxr-xr-x  1 root root 4096 Aug  7 13:03 var
drwx----- 2 root root 4096 Aug  7 13:03 root
drwxr-xr-x  2 root root 4096 Aug  7 13:03 bin
drwxrwxrwt  2 root root 4096 Aug  7 13:03 tmp
drwxr-xr-x  1 root root 4096 Aug 15 07:28 bin
drwxr-xr-x  1 root root 4096 Aug 15 07:28 run
drwxr-xr-x  1 root root 4096 Jan 19 08:47 etc
dr-xr-xr-x 142 root root    0 Jan 19 08:47 proc
dr-xr-xr-x 13 root root    0 Jan 19 08:47 sys
drwxr-xr-x  5 root root 360 Jan 19 08:47 dev
root@372de3e24e55:/#
```

Restart closed container

- We can see in the containers list a status on each container.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2dfc8b1bc8fd	ubuntu	"/bin/bash"	18 minutes ago	Exited (0) 18 minutes ago		vigilant_rubin
ecbf25d5e942	hello-world	"/hello"	20 minutes ago	Exited (0) 20 minutes ago		bold_kilby
372de3e24e55	ubuntu	"bash"	33 minutes ago	Exited (0) 4 seconds ago		linux-container
f6d2e8031bfc	hello-world	"/hello"	41 minutes ago	Exited (0) 41 minutes ago		name-example

- In case that the container status is exit, you need to restart the container before you can go inside it.

```
PS C:\Users\alexk> docker start 372de3e24e55
372de3e24e55
PS C:\Users\alexk> docker exec -it linux-container bash
root@372de3e24e55:/#
```

```
root@372de3e24e55:/# mkdir check
root@372de3e24e55:/# cd check
```

Show options

- We can also see the images that are downloaded to our computer.

```
PS C:\Users\alexk> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
bash                latest             723771701d55       2 months ago      15.2MB
django_project_app  latest             3458e797e92e       4 months ago      141MB
<none>              <none>             bd3f9559f41f       4 months ago      141MB
my-exercise         latest             643b6f287aae       4 months ago      64.2MB
my-ubuntu-image     latest             5140e5890dbc       4 months ago      127MB
<none>              <none>             f2e5f0590bb8       4 months ago      64.2MB
python              3.7-alpine         39fb80313465       4 months ago      98.7MB
ubuntu              latest             a2a15febcd3f       5 months ago      64.2MB
hello-world         latest             fce289e99eb9       12 months ago     1.84kB
PS C:\Users\alexk>
```

- We can delete an image by executing the following command. But we need to delete the containers that are using the image before.

```
docker rmi hello-world
```

- If we will run the command bellow it will remove all our containers.

```
PS C:\Users\alexk> docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
2dfc8b1bc8fd7aa8f9433b5c519d3907e50edf3ceab9090a51770cfaf27a45fc
ecbf25d5e942a3f5ae025297ecc9f282ae42babd80fa3b478e638853154437ef
```

```
Total reclaimed space: 57B
```

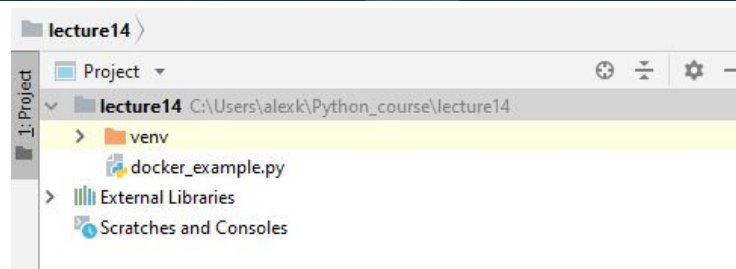
```
PS C:\Users\alexk> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Volumes

- Let's create a container that refers to some local files. This can be done with the -v flag. You can always refer to run --help to get all the details for the syntax. The notation would be `-v [path to local]:[path inside container]`
- The following example is for Windows (where the C:\Users folder is shared with the virtual machine by default). On Mac and Linux you would just use a regular path like /user/admin/Docker.

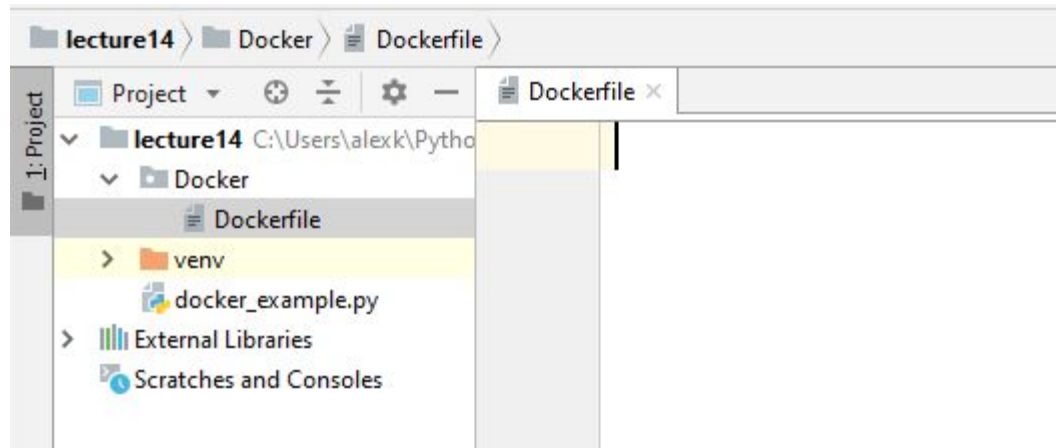
```
PS C:\Users\alexx> docker run -it --name linux-container2 -v C:\Users\alexx\Python_course\lecture14:/my-data ubuntu bash
root@ad51037ee327:/# ls my-data
docker_example.py  venv
root@ad51037ee327:/#
```



- My-data folder is now created in the container and if we will list the content of this folder we will see our prog.py that I created in the original directory.

Images creation

- This is where things get interesting. The dockerfile is your actual "recipe" which defines your images. So far we've used images created by others. As the hello-world container told us, it was looking locally for the hello-world images, did not find it, so it searched online. By online it means the official Docker Hub.
- A Dockerfile is just a file called 'Dockerfile' on your system. Let's create one. I created a new folder on my Desktop and inside this folder, I created a file that is called Dockerfile.

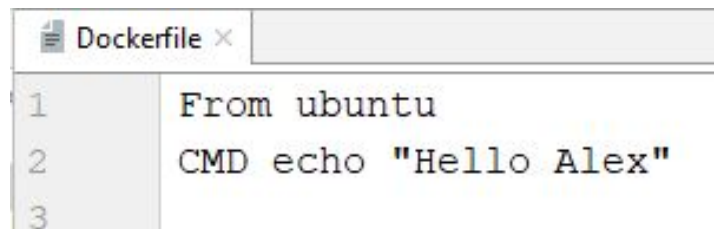


Images creation, continue

- You pretty much always start a Dockerfile based on another container. So your Dockerfile most likely begins with the FROM keyword. After that, you can execute any Linux command you like with the CMD keyword.
- You can find Docker keywords under the following URL.

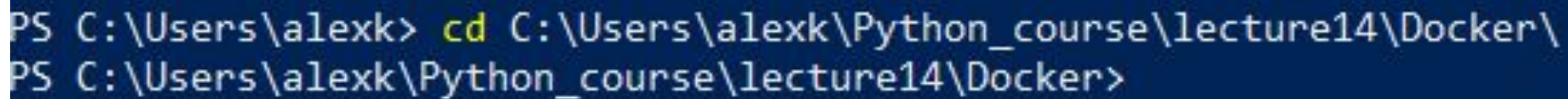
<https://docs.docker.com/engine/reference/builder/>

- Let's write our Dockerfile.



```
Dockerfile x
1 From ubuntu
2 CMD echo "Hello Alex"
3
```

- Let's change our directory to the Docker directory that we just created.



```
PS C:\Users\alexk> cd C:\Users\alexk\Python_course\lecture14\Docker\
PS C:\Users\alexk\Python_course\lecture14\Docker>
```

Images creation, continue

- We can see that inside this directory we have our Dockerfile.
- We will execute the following command.

```
PS C:\Users\alexk\Python_course\lecture14\Docker> docker build -t my-ubuntu-image .
Sending build context to Docker daemon 2.048kB
Step 1/2 : From ubuntu
---> a2a15febcd3
Step 2/2 : CMD echo "Hello Alex"
---> Using cache
---> f2e5f0590bb8
Successfully built f2e5f0590bb8
Successfully tagged my-ubuntu-image:latest
SECURITY WARNING: You are building a Docker image from Windows against a non-Windows Docker host. All files and directories added to build context will have '-rwxr-xr-x' permissions. It is recommended to double check and reset permissions for sensitive files and directories.
PS C:\Users\alexk\Python_course\lecture14\Docker>
```

- Then we can check our images.

```
PS C:\Users\alexk\Python_course\lecture14\Docker> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
bash	latest	723771701d55	2 months ago	15.2MB
django_project_app	latest	3458e797e92e	4 months ago	141MB
<none>	<none>	bd3f9559f41f	4 months ago	141MB
my-exercise	latest	643b6f287aae	4 months ago	64.2MB
<none>	<none>	5140e5890dbc	4 months ago	127MB
my-ubuntu-image	latest	f2e5f0590bb8	4 months ago	64.2MB
python	3.7-alpine	39fb80313465	4 months ago	98.7MB
ubuntu	latest	a2a15febcd3	5 months ago	64.2MB
hello-world	latest	fce289e99eb9	12 months ago	1.84kB

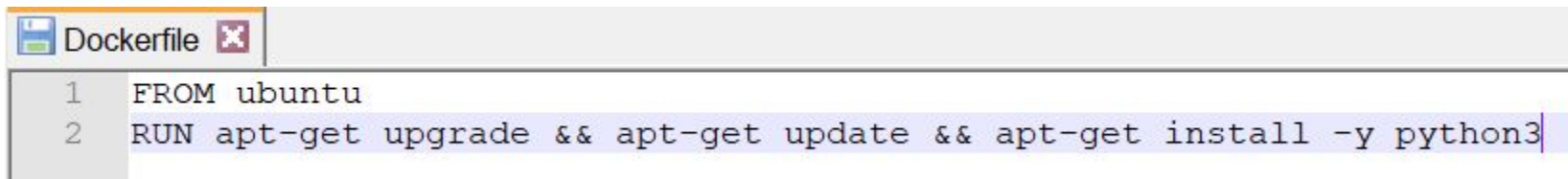
```
PS C:\Users\alexk\Python_course\lecture14\Docker>
```

Images creation, continue

- Let's run our image.

```
PS C:\Users\alexk\Python_course\lecture14\Docker> docker run my-ubuntu-image
Hello Alex
PS C:\Users\alexk\Python_course\lecture14\Docker>
```

- Now let's do something more advanced. Let's say we want to create an Ubuntu container, with python already installed. For that, change your Dockerfile to the following:



The screenshot shows a text editor window titled 'Dockerfile'. It contains two lines of code: '1 FROM ubuntu' and '2 RUN apt-get upgrade && apt-get update && apt-get install -y python3'. The second line is highlighted with a blue selection background.

```
1 FROM ubuntu
2 RUN apt-get upgrade && apt-get update && apt-get install -y python3
```

- Let's rebuild our image.

```
docker build -t my-ubuntu-image .
```


Images creation, continue

- Let's rerun our image and connect to python3.
- Now we run Python commands from the shell.

```
PS C:\Users\alexk\Python_course\lecture14\Docke> docker run my-ubuntu-image
PS C:\Users\alexk\Python_course\lecture14\Docke> docker run -it my-ubuntu-image
root@a5d09dbdcebb:/# python3
Python 3.6.8 (default, Jan 14 2019, 11:02:34)
[GCC 8.0.1 20180414 (experimental) [trunk revision 259383]] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello world")
Hello world
>>>
```

Exercise



Create an Ubuntu based container that prints a text with your personal information.

Image file



- First we will create a file that will declare the image of our container.

```
Dockerfile x
1 From ubuntu
2 CMD echo "My name is Alex"
3
```

- Let's build our image.

```
PS C:\Users\alexk\Python_course\lecture14\Docker> docker build -t ex1-image .
Sending build context to Docker daemon 2.048kB
Step 1/2 : From ubuntu
---> a2a15febcd3
Step 2/2 : CMD echo "My name is Alex"
---> Running in bfa8cf281e2d
Removing intermediate container bfa8cf281e2d
---> ac0fedddf255
Successfully built ac0fedddf255
Successfully tagged ex1-image:latest
SECURITY WARNING: You are building a Docker image from Windows against a non-Windows Docker host. All files and directories added to build context will have '-rwxr-xr-x' permissions. It is recommended to double check and reset permissions for sensitive files and directories.
PS C:\Users\alexk\Python_course\lecture14\Docker>
```

Image run

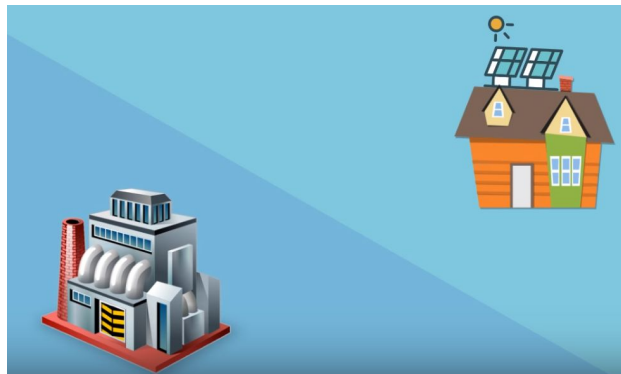


- Run your image.

```
PS C:\Users\alexk\Python_course\lecture14\Docker> docker run -it ex1-image
My name is Alex
PS C:\Users\alexk\Python_course\lecture14\Docker>
```

Cloud Services

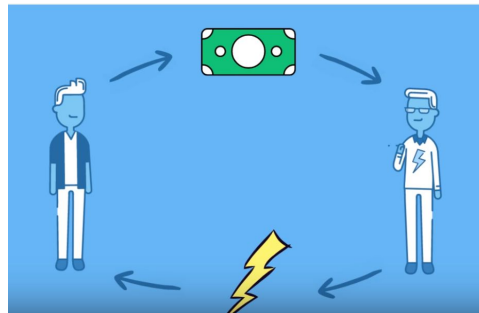
- Let's take an example. Let's say that you are building a house and you have two options for having electricity.
 - 1) Generate your electricity.
 - 2) Buy your electricity from a power company.



- Let's say that you decided to generate your electricity then you will need to invest a lot of money, space and time for this generation.

Cloud Services

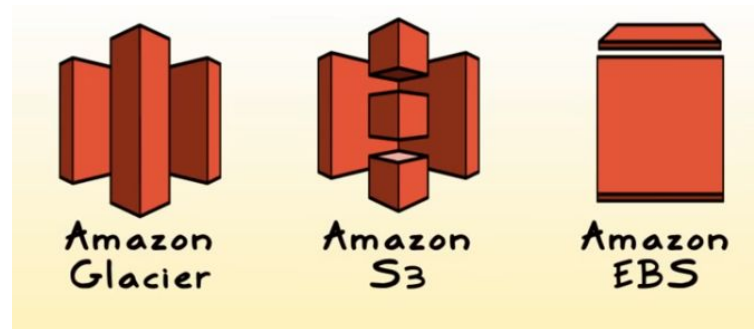
- In case you decided to buy the electricity from the power company then for a monthly fee you will receive all the electricity that you need and all the technical support that you need.



- The works for the High tech industry. A company can buy a lot of servers for their needs. And the cons will be the same as in the previous examples or they can use a cloud service that provides the services that placed on some different sites.

AWS introduction

- As a developer, you are building applications.
- In the AWS you will find a complete set of highly available services.
- You have access to storage.

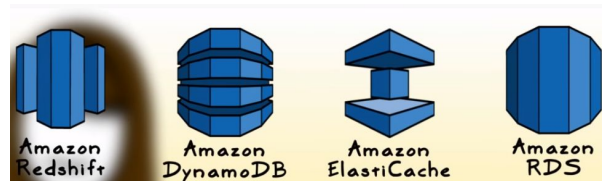


AWS introduction, continue

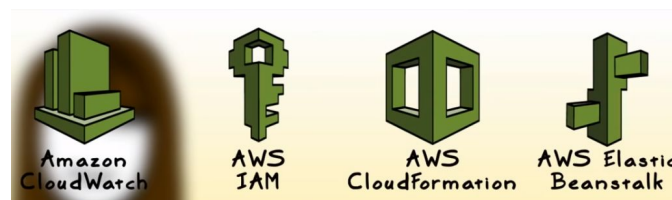
- Low cost compute.



- High performance data bases.



- And tools to manage those resources.



Amazon Web Services

- Let's go to the aws.amazon.com web site.
- We can see it's an amazon tool that allows us different cloud-based services.
- Let's create our account.



Billing window

- As you can see it's mentioned that the 1\$ will be blocked for insurance from your credit card for the next 3-5 days.



When you submit your payment information, we will charge \$1 USD/EUR to your credit card as a verification charge to ensure your card is valid. The amount may show as pending in your credit card statement for 3-5 days until the verification is completed, at which time the charge will be removed. You may be redirected to your bank website to authorize the verification charge.

- We will choose the basic plan at the end of our registration.



Basic Plan

Free



Developer Plan

From \$29/month



Business Plan

From \$100/month

Continue of sign up

- We have an option to create an empty array.

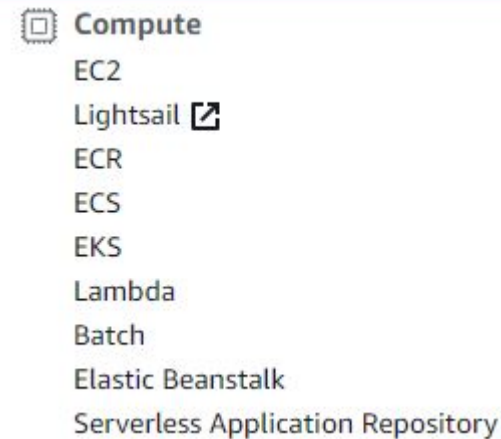
My role is: Student ▼

I am interested in: DevOps ▼

Submit

AWS Management Console

- On this page, we can see all the available services that we have.



- We can also dive into solutions.

